Sec 8.1. Web Application Basics

Web Application Basics

Task 1 - Web Application Overview

Front End components like HTML, CSS, and JavaScript focus on the experience inside the browser. Back End components such as the Web Server, Database, or WAF are the engine under the surface that enable the web application to function.

Answer the questions below

> Which component on a computer is responsible for hosting and delivering content for web applications? web server

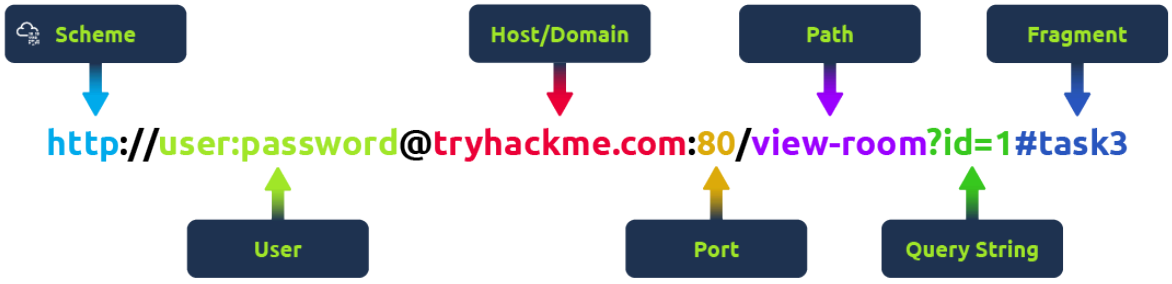
> Which tool is used to access and interact with web applications? web browser

> Which component acts as a protective layer, filtering incoming traffic to block malicious attacks, and ensuring the security of the the web application? web application firewall

Task 3 - Uniform Resource Locator

A Uniform Resource Locator (URL) is a web address that lets you access all kinds of online content—whether it's a webpage, a video, a photo, or other media. It guides your browser to the right place on the Internet.

Anatomy of a URL



- **Scheme:** Defines the communication protocol (HTTP/HTTPS) used to access a website.
- **User:** Contains optional user credentials in a URL, but exposing them creates security risks.
- **Host/Domain:** Identifies the website location and must be checked for phishing and typosquatting.
- **Port:** Specifies the service endpoint used for communication (e.g., 80 for HTTP, 443 for HTTPS).
- **Path:** Identifies the specific resource or page requested from the web server.
- **Query String:** Carries user-controlled parameters and must be secured against injection attacks.
- **Fragment:** Points to a section within a webpage and requires validation to prevent security issues.

Answer the questions below

> Which protocol provides encrypted communication to ensure secure data transmission between a web browser and a web server? HTTPS

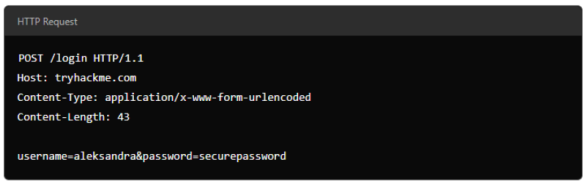
> What term describes the practice of registering domain names that are misspelt variations of popular websites to exploit user errors and potentially engage in fraudulent activities? Typosquatting

> What part of a URL is used to pass additional information, such as search terms or form inputs, to the web server? Query String

Task 4 - HTTP Messages

HTTP messages are packets of data exchanged between a user (the client) and the web server.There are two types of HTTP messages:

- **HTTP Requests**: Sent by the user to trigger actions on the web application.
- **HTTP Responses**: Sent by the server in response to the user's request.



- **Start Line:** Defines the type of HTTP message and provides the request or response details.
- **Headers:** Provide additional metadata and instructions about the HTTP communication.
- **Empty Line:** Separates HTTP headers from the message body.
- **Body:** Contains the actual data being sent in a request or returned in a response.

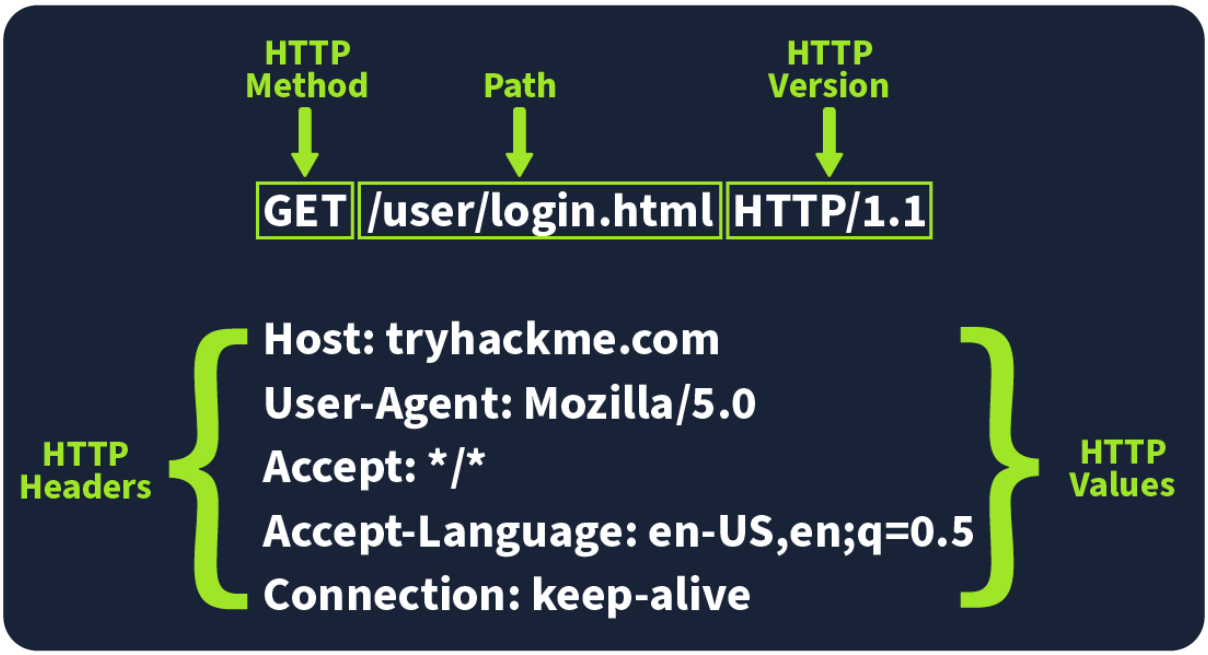
Answer the questions below

> Which HTTP message is returned by the web server after processing a client's request? HTTP response

> What follows the headers in an HTTP message? Empty Line

Task 5 - HTTP Request: Request Line and Methods

An **HTTP request** is what a user sends to a web server to interact with a web application and make something happen.



The **request line** (or start line) is the first part of an HTTP request and tells the server what kind of request it's dealing with. It has three main parts: the **HTTP method**, the **URL path**, and the **HTTP version**.

Example: `METHOD /path HTTP/version`

HTTP Methods: The **HTTP method** tells the server what action the user wants to perform on the resource identified by the URL path. Here are some of the most common methods and their possible security issue:

GET: Used to **fetch** data from the server without making any changes. Reminder! Make sure you're only exposing data the user is allowed to see. Avoid putting sensitive info like tokens or passwords in GET requests since they can show up as plaintext.

POST: Sends data to the server, usually to create or update something. Reminder! Always validate and clean the input to avoid attacks like SQL injection or XSS.

PUT: Replaces or **updates** something on the server. Reminder! Make sure the user is authorised to make changes before accepting the request.

DELETE: Removes something from the server. Reminder! Just like with PUT, make sure only authorised users can delete resources.

Besides these common methods, there are a few others used in specific cases:

PATCH: Updates part of a resource. It's useful for making small changes without replacing the whole thing, but always validate the data to avoid inconsistencies.

HEAD: Works like GET but only retrieves headers, not the full content. It's handy for checking metadata without downloading the full response.

OPTIONS: Tells you what methods are available for a specific resource, helping clients understand what they can do with the server.

TRACE: Similar to OPTIONS, it shows which methods are allowed, often for debugging. Many servers disable it for security reasons.

CONNECT: Used to create a secure connection, like for HTTPS. It's not as common but is critical for encrypted communication.

URL Path: The **URL path** tells the server where to find the resource the user is asking for. For instance, in the URL `https://tryhackme.com/api/users/123`, the path `/api/users/123` identifies a specific user. Attackers often try to manipulate the URL path to exploit vulnerabilities, so it's crucial to:

- Validate the URL path to prevent unauthorised access
- Sanitise the path to avoid injection attacks
- Protect sensitive data by conducting privacy and risk assessments

HTTP Version: The **HTTP version** shows the protocol version used to communicate between the client and server. Here's a quick rundown of the most common ones:

HTTP/0.9 (1991)The first version, only supported GET requests.

HTTP/1.0 (1996)Added headers and better support for different types of content, improving caching.

HTTP/1.1 (1997)Brought persistent connections, chunked transfer encoding, and better caching. It's still widely used today.

HTTP/2 (2015)Introduced features like multiplexing, header compression, and prioritisation for faster performance.

HTTP/3 (2022)Built on HTTP/2, but uses a new protocol (QUIC) for quicker and more secure connections.

Answer the questions below

> Which HTTP protocol version became widely adopted and remains the most commonly used version for web communication, known for introducing features like persistent connections and chunked transfer encoding? HTTP/1.1

> Which HTTP request method describes the communication options for the target resource, allowing clients to determine which HTTP methods are supported by the web server? OPTIONS

> In an HTTP request, which component specifies the specific resource or endpoint on the web server that the client is requesting, typically appearing after the domain name in the URL? URL Path

Task 6 - HTTP Request: Headers and Body

Request Headers allow extra information to be conveyed to the web server about the request. Some common headers are as follows:

Request Header	Example	Description
Host	`Host: tryhackme.com`	Specifies the name of the web server the request is for.
User-Agent	`User-Agent: Mozilla/5.0`	Shares information about the web browser the request is coming from.
Referer	`Referer: https://www.google.com/`	Indicates the URL from which the request came from.
Cookie	`Cookie: user_type=student; room=introtowebapplication; room_status=in_progress`	Information the web server previously asked the web browser to store is held in cookies.
Content-Type	`Content-Type: application/json`	Describes what type or format of data is in the request.

Request Body: In HTTP requests such as POST and PUT, where data is sent to the web server as opposed to requested from the web server, the data is located

inside the HTTP Request Body. The formatting of the data can take many forms, but some common ones are `URL Encoded`, `Form Data`, `JSON`, or `XML`.

- **URL Encoded (application/x-www-form-urlencoded)** A format where data is structured in pairs of key and value where (`key=value`). Multiple pairs are separated by an (`&`) symbol, such as `key1=value1&key2=value2`. Special characters are percent-encoded. ***Example***

```
POST /profile HTTP/1.1
Host: tryhackme.com
User-Agent: Mozilla/5.0
Content-Type: application/x-www-form-urlencoded
Content-Length: 33

name=Aleksandra&age=27&country=US
```

- **Form Data (multipart/form-data)** Allows multiple data blocks to be sent where each block is separated by a boundary string. The boundary string is the defined header of the request itself. This type of formatting can be used to send binary data, such as when uploading files or images to a web server. ***Example***

```
POST /upload HTTP/1.1
Host: tryhackme.com
User-Agent: Mozilla/5.0
Content-Type: multipart/form-data; boundary=----WebKitFormBoundary7MA4YWxkTrZu0gW

----WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="username"

aleksandra
----WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="profile_pic"; filename="aleksandra.jpg"
Content-Type: image/jpeg

[Binary Data Here representing the image]
----WebKitFormBoundary7MA4YWxkTrZu0gW--
```

- **JSON (application/json)** In this format, the data can be sent using the JSON (JavaScript Object Notation) structure. Data is formatted in pairs of name :

value. Multiple pairs are separated by commas, all contained within curly braces { }. ***Example***

```
POST /api/user HTTP/1.1
Host: tryhackme.com
User-Agent: Mozilla/5.0
Content-Type: application/json
Content-Length: 62

{
    "name": "Aleksandra",
    "age": 27,
    "country": "US"
}
```

- **XML (application/xml)** In XML formatting, data is structured inside labels called tags, which have an opening and closing. These labels can be nested within each other. You can see in the example below the opening and closing of the tags to send details about a user called Aleksandra. ***Example***

```
POST /api/user HTTP/1.1
Host: tryhackme.com
User-Agent: Mozilla/5.0
Content-Type: application/xml
Content-Length: 124

<user>
    <name>Aleksandra</name>
    <age>27</age>
    <country>US</country>
</user>
```

Answer the questions below

> Which HTTP request header specifies the domain name of the web server to which the request is being sent? Host

> What is the default content type for form submissions in an HTTP request where the data is encoded as key=value pairs in a query string format? application/x-www-form-urlencoded

> Which part of an HTTP request contains additional information like host, user agent, and content type, guiding how the web server should process the request? Request Headers

Task 7 - HTTP Response: Status Line and Status Codes

The first line in every HTTP response is called the **Status Line**. It gives you three key pieces of info:

1. **HTTP Version**: This tells you which version of HTTP is being used.
2. **Status Code**: A three-digit number showing the outcome of your request.
3. **Reason Phrase**: A short message explaining the status code in human-readable terms.

Status Codes and Reason Phrases: The **Status Code** is the number that tells you if the request succeeded or failed, while the **Reason Phrase** explains what happened. These codes fall into five main categories:

Informational Responses (100-199) These codes mean the server has received part of the request and is waiting for the rest. It's a "keep going" signal.

Successful Responses (200-299) These codes mean everything worked as expected. The server processed the request and sent back the requested data.

Redirection Messages (300-399) These codes tell you that the resource you requested has moved to a different location, usually providing the new URL.

Client Error Responses (400-499) These codes indicate a problem with the request. Maybe the URL is wrong, or you're missing some required info, like authentication.

Server Error Responses (500-599) These codes mean the server encountered an error while trying to fulfil the request. These are usually server-side issues and not the client's fault.

Common Status Codes: Here are some of the most frequently seen status codes:

100 (Continue) The server got the first part of the request and is ready for the rest.

200 (OK) The request was successful, and the server is sending back the requested resource.

301 (Moved Permanently) The resource you're requesting has been permanently moved to a new URL. Use the new URL from now on.

404 (Not Found) The server couldn't find the resource at the given URL. Double-check that you've got the right address.

500 (Internal Server Error) Something went wrong on the server's end, and it couldn't process your request.

Answer the questions below

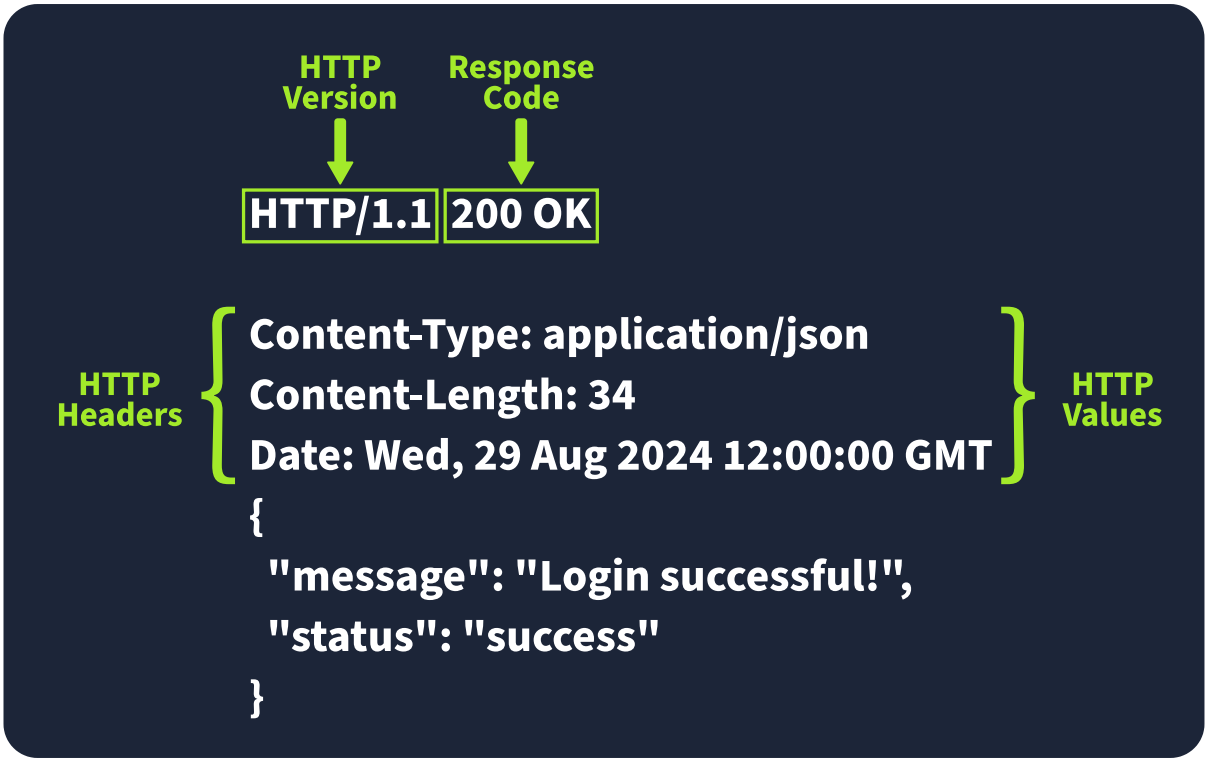
> What part of an HTTP response provides the HTTP version, status code, and a brief explanation of the response's outcome? Status Line

> Which category of HTTP response codes indicates that the web server encountered an internal issue or is unable to fulfil the client's request? Server Error Responses

Which HTTP status code indicates that the requested resource could not be found on the web server? 404

Task 8 - HTTP Response: Headers and Body

When a web server responds to an HTTP request, it includes **HTTP response headers**, which are basically key-value pairs. These headers provide important info about the response and tell the client (usually the browser) how to handle it. Picture an example of an HTTP response with the headers highlighted. Key headers like `Content-Type`, `Content-Length`, and `Date` give us important details about the response the server sends back.



Required Response Headers: Some response headers are crucial for making sure the HTTP response works properly. They provide essential info that both the client and server need to process everything correctly. Here are a few important ones:

- **Date**: Example: `Date: Fri, 23 Aug 2024 10:43:21 GMT`

  This header shows the exact date and time when the response was generated by the server.

- **Content-Type**: Example: `Content-Type: text/html; charset=utf-8`

  It tells the client what kind of content it's getting, like whether it's HTML, JSON, or something else. It also includes the character set (like UTF-8) to help the browser display it properly.

- **Server**: Example: `Server: nginx`

  This header shows what kind of server software is handling the request. It's good for debugging, but it can also reveal server information that might be useful for attackers, so many people remove or obscure this one.

Other Common Response Headers: Besides the essential ones, there are other common headers that give additional instructions to the client or browser and help

control how the response should be handled.

- **Set-Cookie**: Example: `Set-Cookie: sessionId=38af1337es7a8`

  This one sends cookies from the server to the client, which the client then stores and sends back with future requests. To keep things secure, make sure cookies are set with the `HttpOnly` flag (so they can't be accessed by JavaScript) and the `Secure` flag (so they're only sent over HTTPS).

- **Cache-Control**: Example: `Cache-Control: max-age=600`

  This header tells the client how long it can cache the response before checking with the server again. It can also prevent sensitive info from being cached if needed (using `no-cache`).

- **Location**: Example: `Location: /index.html`

  This one's used in redirection (3xx) responses. It tells the client where to go next if the resource has moved. If users can modify this header during requests, be careful to validate and sanitise it—otherwise, you could end up with open redirect vulnerabilities, where attackers can redirect users to harmful sites.

Response Body: The **HTTP response body** is where the actual data lives—things like HTML, JSON, images, etc., that the server sends back to the client. To prevent **injection attacks** like Cross-Site Scripting (XSS), always sanitise and escape any data (especially user-generated content) before including it in the response.

Answer the questions below

> Which HTTP response header can reveal information about the web server's software and version, potentially exposing it to security risks if not removed? Server

> Which flag should be added to cookies in the Set-Cookie HTTP response header to ensure they are only transmitted over HTTPS, protecting them from being exposed during unencrypted transmissions? Secure

> Which flag should be added to cookies in the Set-Cookie HTTP response header to prevent them from being accessed via JavaScript, thereby enhancing security against XSS attacks? HttpOnly

Task 9 - Security Headers

Security Headers: HTTP Security Headers help improve the overall security of the web application by providing mitigations against attacks like Cross-Site Scripting (XSS), clickjacking, and others.

Content-Security-Policy (CSP): A CSP header is an additional security layer that can help mitigate against common attacks like Cross-Site Scripting (XSS). Malicious code could be hosted on a separate website or domain and injected into the vulnerable website. A CSP provides a way for administrators to say what domains or sources are considered safe and provides a layer of mitigation to such attacks. Within the header itself, you may see properties such as `default-src` or `script-src` defined and many more. Each of these give an option to an administrator to define at various levels of granularity, what domains are allowed for what type of content. The use of self is a special keyword that reflects the same domain on which the website is hosted. Looking at an example CSP header:

```
Content-Security-Policy: default-src 'self'; script-src 'self' https://cdn.tryhackme.com; style-src 'self'
```

We see the use of:

- **default-src** which specifies the default policy of self, which means only the current website.
- **script-src** which specifics the policy for where scripts can be loaded from, which is self along with scripts hosted on `https://cdn.tryhackme.com`
- **style-src** which specifies the policy for where style CSS style sheets can be loaded from the current website (self)

Strict-Transport-Security (HSTS): The HSTS header ensures that web browsers will always connect over HTTPS. Let's look at an example of HSTS:

```
Strict-Transport-Security: max-age=63072000; includeSubDomains; preload
```

Here's a breakdown of the example HSTS header by directive:

- **max-age** This is the expiry time in seconds for this setting
- **includeSubDomains** An optional setting that instructs the browser to also apply this setting to all subdomains.

- **preload** This optional setting allows the website to be included in preload lists. Browsers can use preload lists to enforce HSTS before even having their first visit to a website.

X-Content-Type-Options: The X-Content-Type-Options header can be used to instruct browsers not to guess the MIME type of a resource but only use the Content-Type header. Here's an example:

```
X-Content-Type-Options: nosniff
```

Here's a breakdown of the X-Content-Type-Options header by directives:

- **nosniff** This directive instructs the browser not to sniff or guess the MIME type.

Referrer-Policy: This header controls the amount of information sent to the destination web server when a user is redirected from the source web server, such as when they click a hyperlink. The header is available to allow a web administrator to control what information is shared. Here are some examples of Referrer-Policy:

- `Referrer-Policy: no-referrer`
- `Referrer-Policy: same-origin`
- `Referrer-Policy: strict-origin`
- `Referrer-Policy: strict-origin-when-cross-origin`

Here's a breakdown of the Referrer-Policy header by directives:

- **no-referrer** This completely disables any information being sent about the referrer
- **same-origin** This policy will only send referrer information when the destination is part of the same origin. This is helpful when you want referrer information passed when hyperlinks are within the same website but not outside to external websites.
- **strict-origin** This policy only sends the referrer as the origin when the protocol stays the same. So, a referrer is sent when an HTTPS connection goes to another HTTPS connection.

- **strict-origin-when-cross-origin** This is similar to strict-origin except for same-origin requests, where it sends the full URL path in the origin header.

Answer the questions below

> In a Content Security Policy (CSP) configuration, which property can be set to define where scripts can be loaded from? script-src

> When configuring the Strict-Transport-Security (HSTS) header to ensure that all subdomains of a site also use HTTPS, which directive should be included to apply the security policy to both the main domain and its subdomains? includeSubDomains

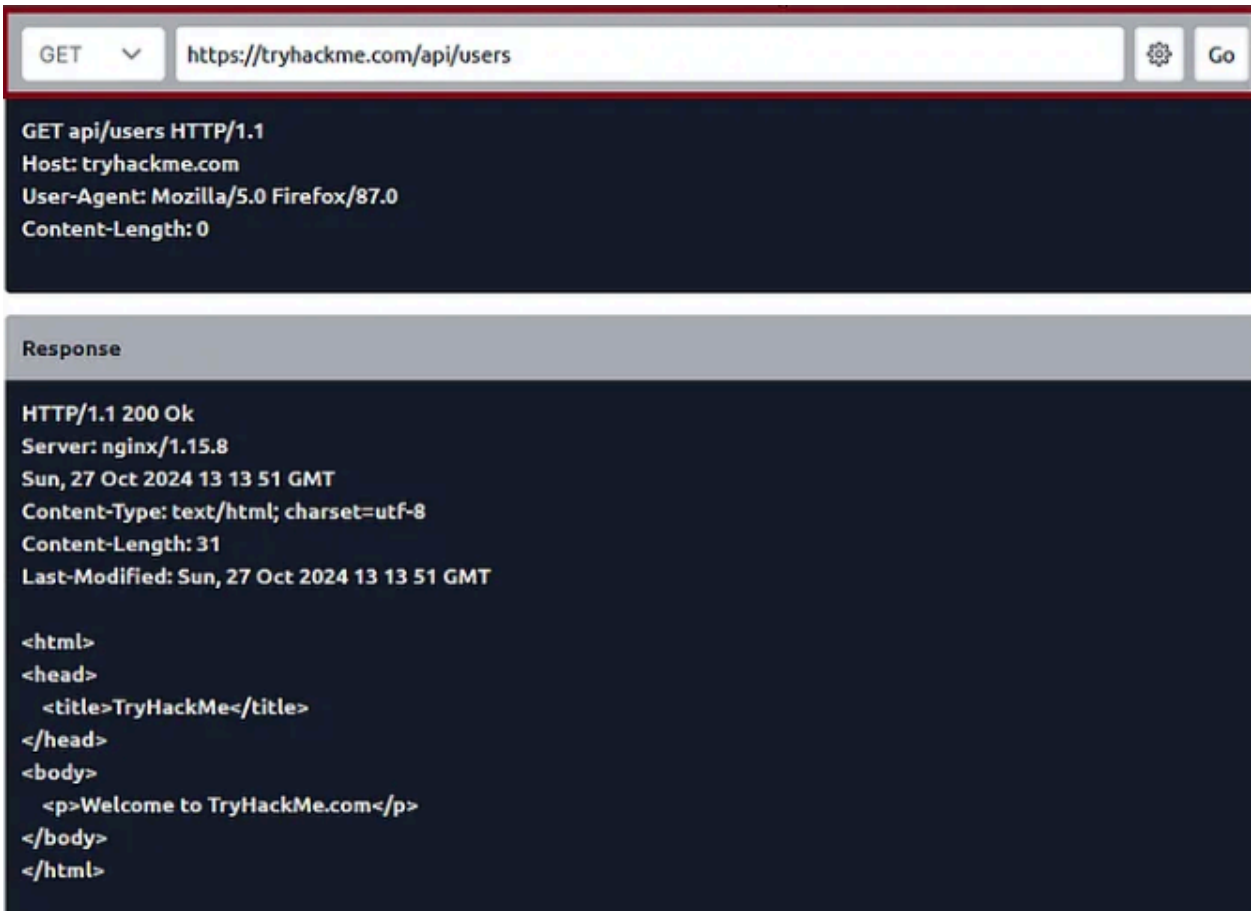
> Which HTTP header directive is used to prevent browsers from interpreting files as a different MIME type than what is specified by the server, thereby mitigating content type sniffing attacks? nosniff

Task 10 - Practical Task: Making HTTP Requests

Click the **View Site** button on the right to launch the static site in Split View.

Answer the questions below

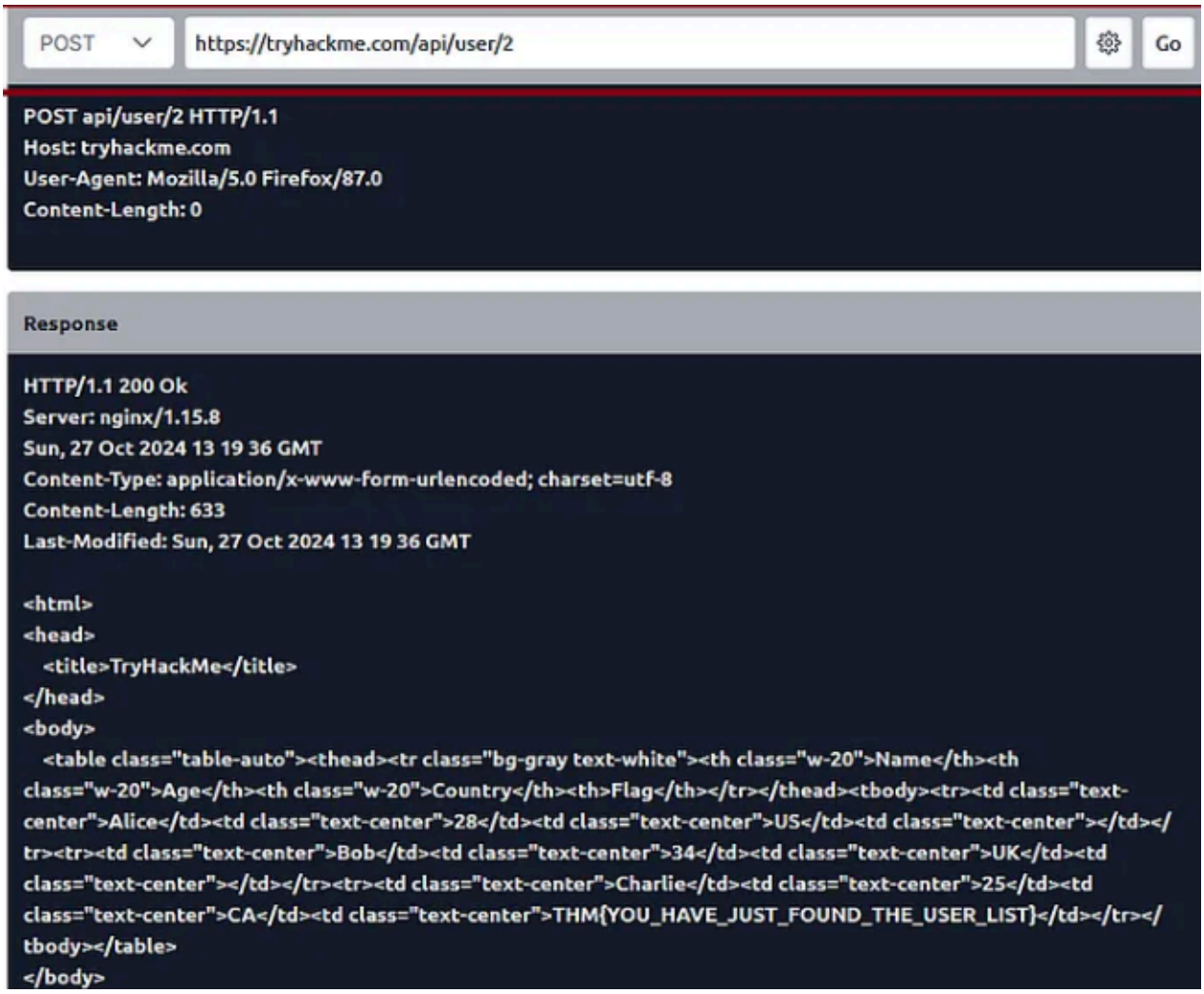
> Make a GET request to /api/users. What is the flag?
> THM{YOU_HAVE_JUST_FOUND_THE_USER_LIST}

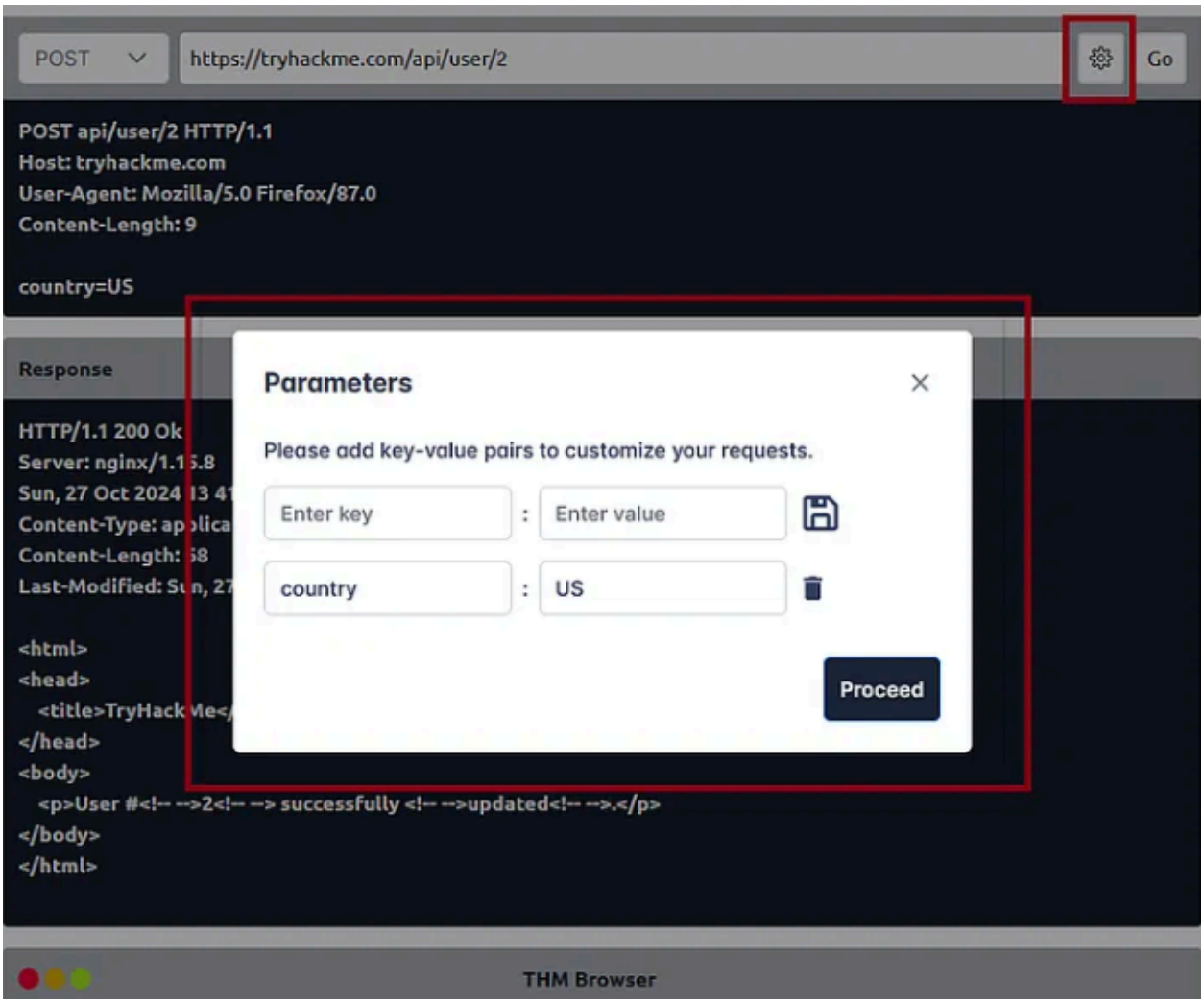

GET
https://tryhackme.com/api/users
Go
GET api/users HTTP/1.1
Host: tryhackme.com
User-Agent: Mozilla/5.0 Firefox/87.0
Content-Length: 0
Response
HTTP/1.1 200 Ok
Server: nginx/1.15.8
Sun, 27 Oct 2024 13 13 51 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 31
Last-Modified: Sun, 27 Oct 2024 13 13 51 GMT
<html>
<head>
<title>TryHackMe</title>
</head>
<body>
Welcome to TryHackMe.com
</body>
</html>

Response

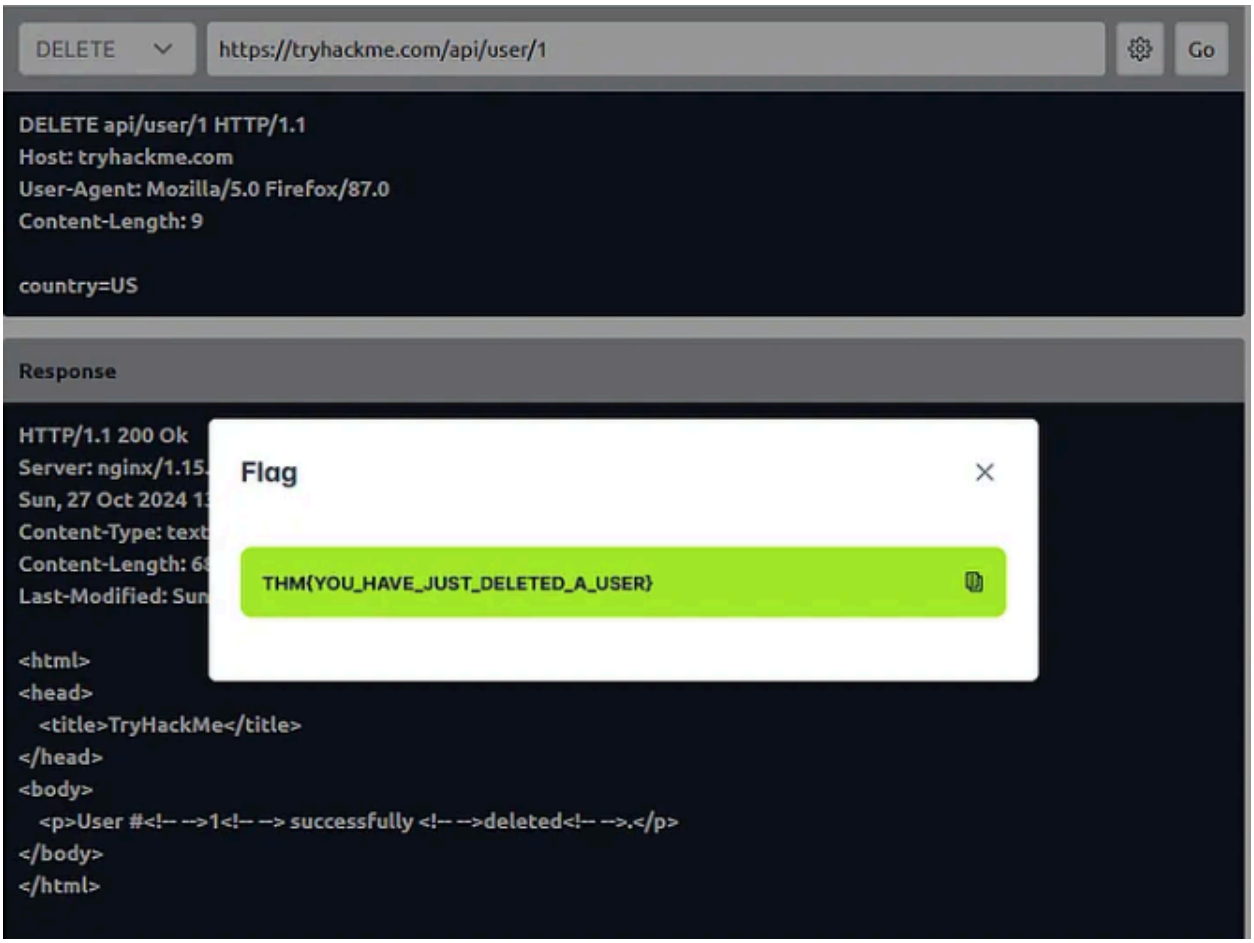
```
HTTP/1.1 200 Ok
Server: nginx/1.15.8
Sun, 27 Oct 2024 13 16 34 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 633
Last-Modified: Sun, 27 Oct 2024 13 16 34 GMT

<html>
<head>
  <title>TryHackMe</title>
</head>
<body>
  <table class="table-auto"><thead><tr class="bg-gray text-white"><th class="w-20">Name</th><th
class="w-20">Age</th><th class="w-20">Country</th><th>Flag</th></tr></thead><tbody><tr><td class="text-
center">Alice</td><td class="text-center">28</td><td class="text-center">US</td><td class="text-center"></td></
tr><tr><td class="text-center">Bob</td><td class="text-center">34</td><td class="text-center">UK</td><td
class="text-center"></td></tr><tr><td class="text-center">Charlie</td><td class="text-center">25</td><td
class="text-center">CA</td><td class="text-center">THM{YOU_HAVE_JUST_FOUND_THE_USER_LIST}</td></tr></
tbody></table>
</body>
</html>
```

> Make a POST request to /api/user/2 and update the country of Bob from UK to US. What is the flag? THM{YOU_HAVE_MODIFIED_THE_USER_DATA}


POST
https://tryhackme.com/api/user/2
Go
POST api/user/2 HTTP/1.1
Host: tryhackme.com
User-Agent: Mozilla/5.0 Firefox/87.0
Content-Length: 0
Response
HTTP/1.1 200 Ok
Server: nginx/1.15.8
Sun, 27 Oct 2024 13 19 36 GMT
Content-Type: application/x-www-form-urlencoded; charset=utf-8
Content-Length: 633
Last-Modified: Sun, 27 Oct 2024 13 19 36 GMT
<html>
<head>
<title>TryHackMe</title>
</head>
<body>
<table class="table-auto"><thead><tr class="bg-gray text-white"><th class="w-20">Name</th><th class="w-20">Age</th><th class="w-20">Country</th><th>Flag</th></tr></thead><tbody><tr><td class="text-center">Alice</td><td class="text-center">28</td><td class="text-center">US</td><td class="text-center"></td></tr><tr><td class="text-center">Bob</td><td class="text-center">34</td><td class="text-center">UK</td><td class="text-center"></td></tr><tr><td class="text-center">Charlie</td><td class="text-center">25</td><td class="text-center">CA</td><td class="text-center">THM{YOU_HAVE_JUST_FOUND_THE_USER_LIST}</td></tr></tbody></table>
</body>



> Make a DELETE request to /api/user/1 to delete the user. What is the flag?
> THM{YOU_HAVE_JUST_DELETED_A_USER}



Task 11 - Conclusion

You have learned a bit more about:

- What components are involved in web applications
- The structure of the Uniform Resource Locator (URL)
- What are HTTP messages, requests, headers and responses
- The importance of Security headers